TenderAI

Architecture of an explainable AI agent for Kazakhstan public procurement analytics

Scope: 27 customer organisations, 2024–2026, data from goszakup.gov.kz OWS v3
Stack: Python 3.11+, PostgreSQL 16, FastAPI, Ollama with Qwen3.5 9B (on-prem GPU)
Languages: questions and answers in Russian and Kazakh
Document date: 18 October 2026, written from the current repository state
Data as of: 13 October 2026, 14:02 UTC (latest incremental sync)

Contents

1. Overview and design principles
2. System architecture
3. Repository layout
4. Data source: OWS v3
5. Ingestion
6. Storage schema
7. ETL: raw to core to marts
8. Analytics methods
9. The agent
10. API, web chat and worker
11. Deployment
12. Security
13. Testing and evaluation
14. Risks and limitations
15. Operating the system

1. Overview and design principles

TenderAI is an analytical agent over Kazakhstan's public procurement portal, goszakup.gov.kz. It loads procurement plans, tender announcements, lots, contracts and organisations for 27 customer organisations (identified by BIN, the Kazakh business ID) from 2024 to 2026. Users ask questions in Russian or Kazakh, and every answer cites the source records with portal links, the sample size N, the method, a confidence score and the limitations.

The agent handles six kinds of question, called *intents*:

Intent	Example question	What it returns
search	Show purchases of A4 paper by customer X	Matching lots and contract lines with links
compare	Compare spending on communication services between agencies in 2024; compare 2024 and 2025	Per-customer or per-year totals and prices
analytics	What was the average diesel price in 2025?	Price statistics for one product code
price_anomaly	Find purchases with a price deviation above 25%	Observations far from the peer weighted average
fair_price	Is the price of lot No. 80800385-OI6 fair?	Fair Price, band, deviation, verdict, confidence, and whether the lot's quantity is normal for the customer
volume_anomaly	Find purchases whose volume jumped versus earlier years	Customer × product series with atypical growth

Design principles

> **The LLM never calculates.** SQL and Python produce every figure. The LLM does two narrow jobs: it classifies the question into a JSON schema, and it writes a 1–3 sentence verdict. Every number, BIN and date in that verdict is checked against the tool output before it is shown. If the check fails twice, or the LLM is offline, a template verdict is used instead.

- **Explainability by construction.** Five of the six answer sections are rendered directly from the tool result, never by the LLM. Each answer lists parameters, N, method, confidence, limitations and the Top-K object IDs with links.
- **Raw data is kept verbatim.** API payloads are stored as JSONB, so cleaning rules can change and the derived tables can be rebuilt in about a minute without calling the API again.
- **Idempotent, restartable ingestion.** Upserts are keyed by (entity, id) with a content hash, so re-running any load is safe.
- **Fully on-prem.** The LLM runs locally in Ollama on the GPU and has no internet, SQL or tool access. The only outbound calls are to the OWS API, and only during ingestion.
- **Graceful degradation.** Without the LLM the system still answers using rule-based routing, customer names found in the question text, and template verdicts.
- **Refuse rather than guess.** A customer outside the 27, a year outside 2024–2026, an off-topic question or a fairness question without a lot number gets a 'no data' answer that states the reason, never figures for something else.

2. System architecture

The system has four layers. External sources feed an ingestion layer that writes raw payloads into PostgreSQL. An ETL step turns raw data into typed, linked **core** tables and then into analytical **mart** tables. The agent reads only core and mart through a read-only database role, and the local LLM sits beside it.

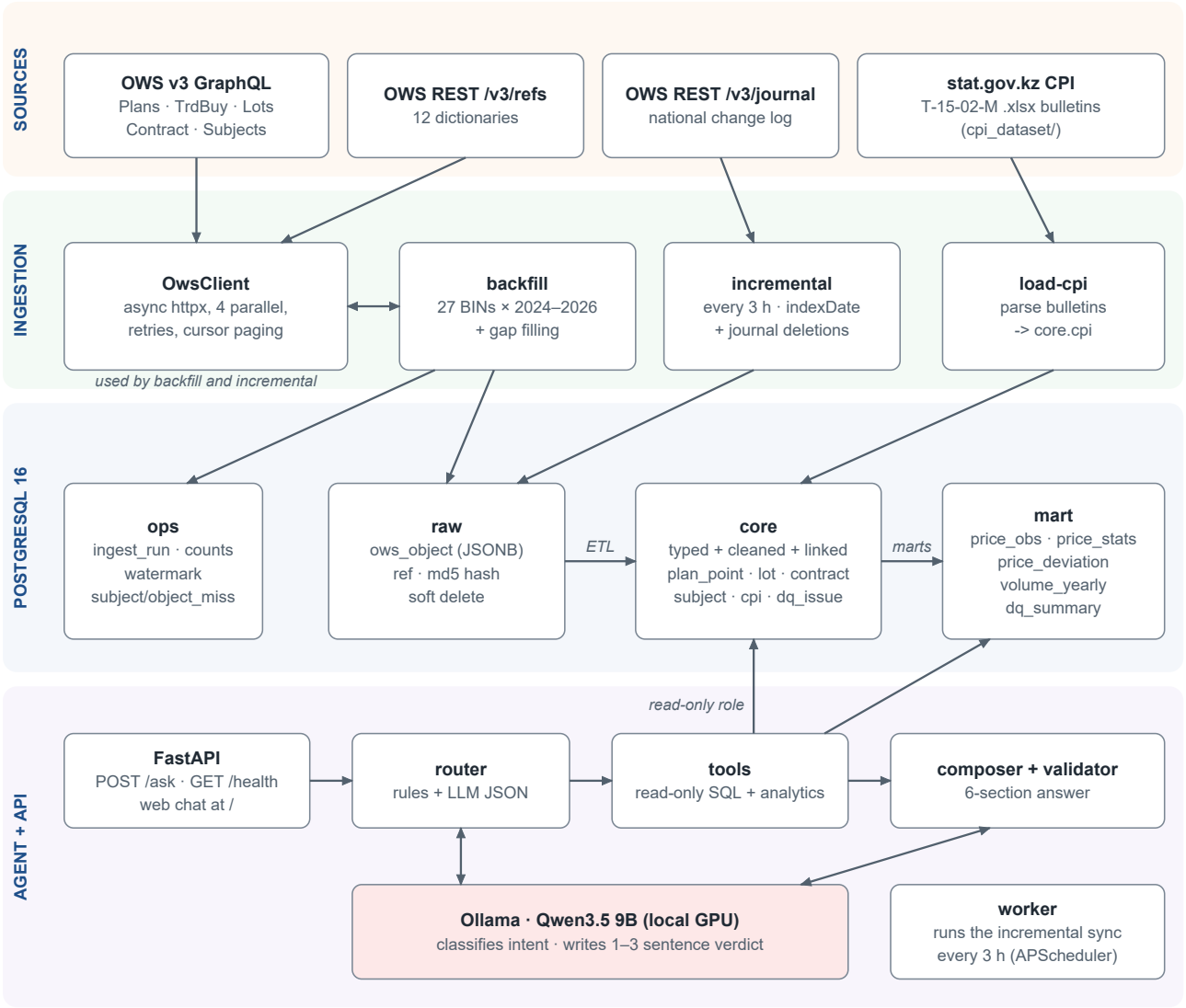


Figure 1. Components and data flow. Arrows show the direction data moves.

The pipeline in one line

```
OWS v3 GraphQL/REST --> ingest (async, retries, idempotent upserts) --> raw (JSONB)
       `-- journal (deletions only, action=D) --> incremental sync every 3 h (lag < 24 h)
raw --> ETL (SQL + Python: units, names, KATO, ENSTRU, DQ rules) --> core --> marts
marts --> deterministic tools (Fair Price, anomalies, volumes, stats) --> local LLM verdict
      --> fact + claim validator --> templated 6-section answer --> FastAPI /ask + web chat
```

Glossary

Term	Meaning
OWS v3	The public data API of goszakup.gov.kz (version 3), at ows.goszakup.gov.kz/v3
BIN / IIN	12-digit business ID / individual ID (sole traders are registered by IIN)
ENSTRU	The national classifier of goods, works and services; every plan line carries an ENSTRU code

Term	Meaning
KATO	The administrative-territorial classifier; the first 2 digits give the region (oblast or major city)
MKEI	The unit-of-measure classifier (kg, litre, piece, pack...)
CPI	Consumer price index from the Bureau of National Statistics (stat.gov.kz), used to adjust for inflation
DQ	Data quality: rules in etl/dq.sql that flag errors (excluded) and warnings (kept, lower confidence)

3. Repository layout

Path	Role
tenderai/config.py	Settings from environment / .env (pydantic-settings); the 27 target BINs and years 2024–2026
tenderai/cli.py	Entry point: migrate, backfill, incremental, etl, load-cpi, worker, ask, serve
tenderai/ows/client.py	Async OWS client: GraphQL and REST, retries, cursor paging, error detection
tenderai/ingest/	queries.py (GraphQL field lists), backfill.py, incremental.py, store.py (raw upserts, run log)
tenderai/db/	migrate.py and numbered SQL migrations 001–010 (schemas, roles, CPI, miss tables)
tenderai/etl/	run.py (orchestrator), transform.sql, dq.sql, marts.sql, units.py, lot_names.py, cpi.py
tenderai/analytics/	fair_price.py, price_index.py, stats.py (robust statistics), volumes.py
tenderai/agent/	agent.py (orchestration), router.py, tools.py, llm.py, validator.py, composer.py, i18n.py
tenderai/api/	main.py (FastAPI) and static/chat.html (web chat)
tenderai/worker/	scheduler.py: APScheduler running the incremental sync every 3 hours
deploy/	Dockerfile, docker-compose.yml, k8s/ manifests (namespace and secrets, Postgres, Ollama, app)
tests/	93 unit tests (router, name matching, product variants, validator, composer, Fair Price, volumes, index, names, eval leakage) and OWS fixtures
eval/	Held-out set (23 fresh questions), corpus (54 hand-picked questions), golden set (40 templated) and paraphrases (24), all with SQL-derived reference answers; the evaluation runner, leakage check and saved reports
docs/	architecture, schema, analytics, data_scope, examples, risks; docs/report builds the PDFs
cpi_dataset/	Official stat.gov.kz T-15-02-M monthly CPI bulletins (.xlsx), loaded by load-cpi

The Python and SQL code is about 5,500 lines. The largest modules are the agent tools (tools.py), the answer composer and the Fair Price engine. TenderAI_Report.pdf in the repository root explains how the application meets each requirement of the assignment.

4. Data source: OWS v3

OWS v3 exposes three interfaces. GraphQL is the primary ingest path: it carries complete lot data (names, quantities, ENSTRU lists, delivery KATO) and supports server-side filters by BIN, year and `indexDate`. REST is used only for dictionaries and the change journal.

Interface	Used for	Paging
POST /v3/graphql	Plans, TrdBuy, Lots, Contract (+ ContractUnits), Subjects	max 200 rows; cursor after = lastId
GET /v3/refs/{name}	12 dictionaries: units, KATO, trade methods, statuses, months, finance sources...	500 rows per page
GET /v3/journal	National change log; used only to detect deletions (action = D)	500 rows per page

Entities and why each is needed

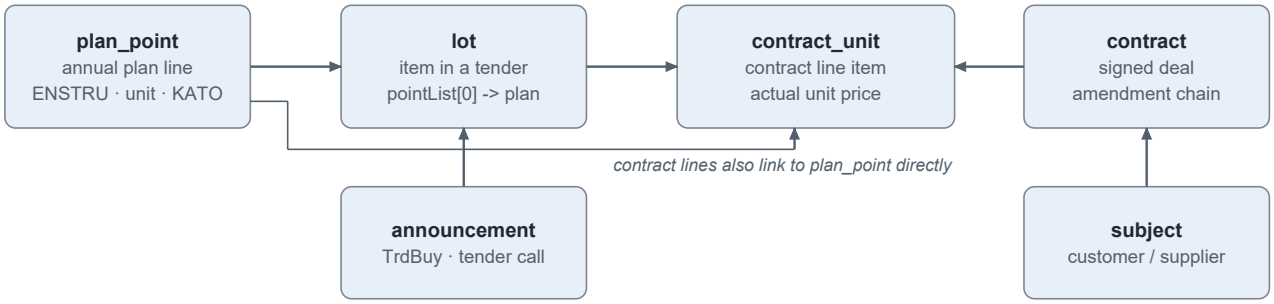


Figure 2. How procurement records link together: plan line -> lot -> contract line, with announcements, contracts and organisations around them.

Entity	Filter used	Why it matters
Plans (plan_point)	customer BIN + plan year 2024–2026	ENSTRU code, unit, planned price and quantity, delivery KATO. Basis for volume anomalies
TrdBuy (announcement)	organiser BIN + publish date in period	Publication date, trade method, status, portal link
Lots (lot)	customer BIN only (no date filter in the API)	Announced price and quantity; links to plan line and announcement
Contract + ContractUnits	customer BIN + sign date in period	Actual unit prices without VAT, supplier, status, amendment chain
Subjects	BIN, or IIN for sole traders	Names, registered address KATO, supplier flags
RefEnstru (nested)	comes inside each plan record	ENSTRU dictionary; the REST endpoint returns 'not found' in v3

API behaviour the code relies on

- The token in the public documentation is a sample and returns 403. A working token is issued separately and lives only in `OWS_TOKEN`.
- Errors can arrive inside an HTTP 200 body, so the client checks the body as well as the status.
- Date filters accept `YYYY-MM-DD` or ISO with a `T`; timestamps are local time (UTC+5).
- Some REST text fields are double-encoded UTF-8; GraphQL text is correct.
- Timeouts and 5xx errors come in bursts; exponential backoff absorbs them.
- The journal is national and its volume swings: about 105,000 changes on a normal day, 4.6 million on 12 October 2026 when the portal re-indexed plans in bulk. The incremental sync re-queries only the 27 BINs and reads the journal just for deletions, filtered on the server with `action=D` (746 pages that day instead of 9,276). `date_to` is exclusive.

5. Ingestion

5.1 OWS client (tenderai/ows/client.py)

- Async `httpx` client with a bearer token from settings. The token is a `SecretStr`, and httpx request logging is silenced so URLs and headers never reach logs.
- A semaphore limits concurrency to 4 requests (`OWS_CONCURRENCY`).
- Retries up to 6 times on network errors, 429 and 5xx, with exponential backoff (1 s doubling to 30 s, plus jitter). 401/403 raise a clear 'check OWS_TOKEN' error immediately.
- `graphql_paginate` walks pages using `extensions.pageInfo.lastId` and fails loudly if the cursor stops advancing. `rest_paginate` follows `next_page`.

5.2 Backfill (tenderai/ingest/backfill.py)

The full initial load, about 25 minutes. It runs in this order:

- **1. Dictionaries**: the 12 REST reference tables into `raw.ref`.
- **2. Per-BIN entity loads**: plans, announcements, lots and contracts for each of the 27 BINs. For every (entity, BIN) the API's reported total is stored next to the rows fetched in `ops.ingest_count`.
- **3. Gap filling**: plan lines referenced by stored lots or contract lines but outside the BIN/year filter (about 134,000), and announcements organised by third parties for in-scope lots, are fetched by ID.
- **4. Subjects**: the 27 customers plus every supplier found on a contract.

IDs that the API no longer returns (for example superseded plan versions) are recorded in `ops.object_miss`, and BIN/IINs with no Subjects result in `ops.subject_miss`. Both are skipped for 7 days rather than re-requested on every run.

5.3 Incremental sync (tenderai/ingest/incremental.py)

Runs every 3 hours from the worker (or a Kubernetes CronJob). The freshness target is a data lag under 24 hours.

- **Changed records**: for each of the 4 entities and 27 BINs, re-run the backfill filter plus `indexDate >= watermark - 2 h`. The 2-hour overlap guards against late indexing.
- **Deletions**: page through `/v3/journal?action=D` for the same window (8 pages in parallel, progress logged). Deleted objects are soft-deleted (`is_deleted = true`), except objects whose stored version was re-indexed inside the window: this run has just fetched them live, so the deletion predates a re-publication.
- **Follow-ups**: fetch newly referenced plan lines, announcements and suppliers.
- **Watermarks**: update `incremental` (next window start) and `journal` (read by /health for freshness), then rebuild core and marts.

If there is no watermark yet, the start time of the first successful backfill is used; with no backfill at all, the sync refuses to run. The latest run (13 October 2026) covered two days of the journal, 1,644 deletion pages, in about an hour, soft-deleting 503 lots and 20 plan lines; the ETL rebuild took 52 seconds.

5.4 Raw store (tenderai/ingest/store.py)

- Upserts into `raw.ows_object` keyed by (entity, id). An md5 of the canonical JSON is stored, and the row is only rewritten when the hash differs.
- Rows repeated across page boundaries within one page batch are de-duplicated (last wins).
- `RunLog` records each run in `ops.ingest_run` with kind, status, stats and error.

5.5 Current volumes (13 October 2026)

Entity	Rows
Plan line versions (in scope and referenced from other years or BINs)	152,246
Announcements	65,835
Lots, all years / in scope	273,110 / 117,106
Contract versions / contract line items	36,831 / 105,324
Organisations (customers and suppliers)	8,849
Priced observations (mart.price_obs)	78,678
Data-quality issues recorded	236,601

6. Storage schema

PostgreSQL 16 holds four schemas, each with one job. Migrations live in `tenderai/db/sql` and are applied in order by `tenderai migrate`.

Schema	Purpose	Written by
raw	Exact API payloads (JSONB), keyed by (entity, id), with change hash and soft-delete flag; plus dictionaries	ingest
core	Typed, cleaned, linked entities, rebuilt deterministically from raw in one transaction	etl/transform.sql, dq.sql
mart	Analytical tables the agent reads: prices, deviations, statistics, volumes, DQ summary	etl/marts.sql
ops	Ingestion runs, per-BIN reconciliation, watermarks, miss lists	ingest

Core tables

Table	Grain	Key columns and notes
core.plan_point	plan line version	is_latest per rootrecord_id; enstru_code; unit_code -> base_unit; count_base, price_base; kato_code + kato_source; url
core.announcement	announcement	number_anno, org_bin, publish_date, status, url
core.lot	lot	plan_point_id (pointList[0]); unit_price_base; in_scope (plan or publish year in 2024–2026); name_clean
core.contract	contract version	is_latest (not superseded by a child); sum without VAT; supplier_biin; url
core.contract_unit	contract line	item_price without VAT; unit_price_base; quantity_base; lot_id; plan_point_id
core.subject	organisation	biin, name_ru, kato_code, is_target
core.ref_unit / ref_kato / ref_enstru / ref_status	dictionaries	ref_unit has base_unit and factor from units.py
core.cpi	month × region × category	Official CPI level (Dec 2020 = 100) and month-on-month change
core.dq_issue	(entity, obj_id, rule)	severity error / warning; detail JSONB
core.target_customer	BIN	The assignment's 27 customers

Marts

Mart	Grain	Used by
mart.price_obs	one priced observation: a contract line, or a lot with no contract	all price tools, Fair Price, search
mart.price_stats	ENSTRU × unit × (all years / per year)	price_stats tool
mart.price_deviation	observation + leave-one-out peer weighted average + robust z	price_anomalies tool
mart.volume_yearly	customer × ENSTRU × unit × year (planned)	volume_anomalies tool
mart.dq_summary	entity × rule	confidence blocks, monitoring

Access control

- `tenderai` owns everything and is used by ingest and ETL.
- `tenderai_agent` (migration 004) can only SELECT from core and mart, with `default_transaction_read_only = on` and `statement_timeout = 15s`. All agent tools connect as this role through `AGENT_DATABASE_URL`.

7. ETL: raw to core to marts

`tenderai/etl/run.py` rebuilds everything inside **one transaction**, so readers never see a half-built core. A full run takes about a minute.

Step	What happens
1. Units	Load MKEI units from raw.ref and map each to (base unit, factor) with units.py: g/kg/t -> kg, ml/L/m³ -> L, m/km -> m, thousand pcs -> pcs. Packaging units stay as their own base.
2. Targets	Fill core.target_customer from TARGET_BINS in config.py.
3. transform.sql	Build dictionaries, subjects, plan points (latest version per rootrecord), announcements, lots (in_scope flag), contracts (final version of each amendment chain) and contract lines. Infer a missing unit from the ENSTRU code only when at least 90% of at least 3 plan points agree.
4. Name cleaning	lot_names.py normalises plan and lot names into name_clean: lot numbering, years, 'согласно технической спецификации', quotes, enumeration, Latin look-alike letters, and procedural lead words ('Приобретение', 'Закуп'). Contract lines inherit the plan's clean name.
5. dq.sql	Apply data-quality rules into core.dq_issue: errors (non-positive price or quantity, missing unit or ENSTRU) are excluded from statistics; warnings are kept and lower confidence.
6. marts.sql	Build price_obs, constant-price columns, price_stats, price_deviation, volume_yearly and dq_summary.
7. ANALYZE	Refresh planner statistics after the rebuild.

Key cleaning rules

- **Prices** are always without VAT, in KZT, per base unit (`price_base = price / factor`).
- **Observation priority**: a contract line (actual price) replaces the lot it came from; lots without a contract count as announced prices.
- **Versions**: only the final contract version and the latest non-deleted plan version are used.
- **Delivery location** falls back lot -> plan -> announcement -> customer's registered address, and `kato_source` records which one was used (about 70% of plan lines have no delivery KATO).
- **ENSTRU formats**: the old 37-character KPVED-based and new `NNNNNN.NNN.NNNNNN` codes are matched exactly first, then by prefix within the same format.

CPI loader (tenderai/etl/cpi.py)

`tenderai load-cpi cpi_dataset` parses the official stat.gov.kz T-15-02-M monthly bulletins (33 workbooks covering January 2024 to September 2026). Sheet 1 gives national CPI and sheet 2 regional CPI, each in four categories: total, food, non-food and paid services. Columns are located by header text because layouts differ between years. The fixed-base level (December 2020 = 100) is stored, so no chaining error builds up. Region names map to 2-digit KATO codes (20 regions plus KZ for national). A CSV with a monthly index is accepted as an alternative source.

8. Analytics methods

All metrics live in `tenderai/analytics` and `tenderai/etl/marts.sql`. `stats.py` provides the shared robust statistics: weighted quantiles and median, MAD, robust z = (x - median) / (1.4826 · MAD), and IQR bounds.

8.1 Fair Price (analytics/fair_price.py)

For a target lot with ENSTRU code *e*, base unit *u*, delivery city *c* in region *r*, date *t* and unit price *p*:

Step	Rule
1. Comparable set	From mart.price_obs, take the first level with N ≥ 5: same city -> same region -> national -> coarser ENSTRU prefix. Always other customers only, within 18 months, price > 0. If no level reaches 5, use the largest set and mark the answer preliminary.

Step	Rule
2. Time adjustment	$p' = p_i \cdot I(t) / I(t_i) \cdot S(m_t) / S(m_i)$. I is the official CPI of the comparable's region and the item's CPI category (food, non-food, services, total by ENSTRU division). Without CPI, an internal procurement price index is used. S is a seasonal index, applied only when N ≥ 30 across ≥ 6 months.
3. Regional coefficient	$K(r) = w \cdot \text{median}(p' \text{ in } r) / \text{median}(p') + (1 - w)$, with $w = n_r / (n_r + 10)$. Prices are moved to the target region; shrinkage stops tiny regions producing extreme coefficients.
4. Fair price	FairPrice = median of adjusted prices; fair band = [Q1, Q3]. The quantity-weighted mean is reported alongside.
5. Verdict	deviation = (p - FairPrice) / FairPrice. Overpriced if > +30% or z > 3; underpriced (dumping risk) if < -30% or z < -3; fair if inside the band or within ±10%; otherwise elevated or below market.
6. Confidence	0.35 · min(1, ln N / ln 30) + 0.25 · 1/(1 + IQR/median) + 0.20 · level score + 0.10 · (1 - DQ rate) + 0.10 · region known. High ≥ 0.70, medium ≥ 0.45, otherwise low; capped at low when N < 5.

Quantity of the lot. The fairness answer also judges the lot's quantity: the customer's planned volume of that ENSTRU × unit in the lot's year against the median of its earlier years (mart.volume_yearly). Growth above 1.5× is reported as atypical; without an earlier year the answer says there is nothing to compare with.

8.2 Price anomalies (mart.price_deviation)

- Every observation is converted to constant prices with the official CPI of its own region and category.
- The baseline is the quantity-weighted average of the **other** observations of the same ENSTRU × unit (leave-one-out), so an outlier cannot pull its own baseline up and hide itself. Groups need at least 3 peers.
- deviation = price / peer average - 1. The 30% rule alone flags about 30% of goods positions because ENSTRU codes are broad, so each observation also gets a robust z of log price within its group.
- **Confirmed** anomalies satisfy both the threshold and z > 3. They are listed first, ranked by estimated overpayment (amount - peer average × quantity).
- Services and works priced per '1 service' are excluded by default, since a per-unit price is not comparable.
- 'Below market' questions ("ниже рынка", "демпинг") use the same method in the other direction (deviation below −threshold, z < −3), ranked by the deepest discount.

8.3 Volume anomalies (analytics/volumes.py)

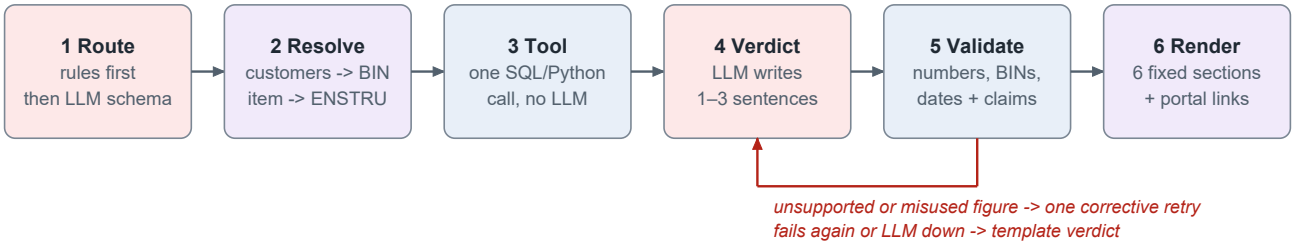
- Series: customer × ENSTRU × base unit × year, using **planned** quantity from the latest plan versions.
- baseline = median of previous years; flagged when ratio = qty / baseline > 1.5 and, with ≥ 2 previous years, the robust z of log quantity against the customer's own history is > 3.
- **Confirmed** tier: flagged, statistically extreme (z > 3, or ≥ 3× growth with one previous year) and material (extra planned spend ≥ 1,000,000 KZT). On current data that is 482 of 12,898 series (3.7%).
- An Isolation Forest (scikit-learn) on [ln ratio, ln amount ratio, ln unit-price ratio, number of plan points] gives a secondary score that is shown but never flags anything by itself.

8.4 Data quality and confidence in every answer

Each answer reports the DQ exclusion rate of the entities it uses; error rows never enter statistics. Fair Price has its own confidence (step 6). Every other answer carries an aggregate confidence: 0.6 · min(1, ln N / ln 30) + 0.4 · (1 − DQ exclusion rate), labelled high ≥ 0.70, medium ≥ 0.45, low otherwise, and always low under 5 objects.

9. The agent

`tenderai/agent/agent.py` orchestrates one question in six steps. Timings for routing, the tool and the verdict are returned with every answer.



Shaded red = steps that call the local LLM. Every other step is deterministic.

Figure 3. Lifecycle of one question through Agent.ask().

9.1 Router (agent/router.py)

- **Rules always run first.** Language detection (Kazakh-specific letters or at least 2 Kazakh keywords) and regexes for hard facts: 12-digit BINs, ENSTRU codes in both formats, lot numbers (Russian and Kazakh word order), announcement numbers, percentage thresholds and 'top-N'. Years: '2025 compared with 2024' means 2025, a range keeps all years, and a year outside 2024–2026 is refused. A unit named in the question ('в пачках') filters by that unit; 'ниже рынка' / 'демпинг' sets the anomaly direction; questions with nothing about procurement are flagged as off-topic. Keyword rules give a fallback intent.
- **Then the LLM** fills a JSON schema (intent, item_text, customer_text, year, threshold_pct, top_k) using Ollama's constrained output, with a few-shot Russian system prompt. Kazakh item names are translated to Russian.
- **Hard facts override the LLM.** A detected lot number in a fairness question forces fair_price; a fairness intent without a lot number falls back to the rules; ranking questions ('какой заказчик больше всех') and 'сравни 2024 и 2025' are comparisons. Generic words such as 'goods' or 'purchases' are discarded as item names, and a lot number copied into the customer field is removed.
- If the LLM fails or returns invalid JSON, the rule route is used unchanged.

9.2 Resolve and dispatch

- **Customers**: several names are split ('A и B', 'A; B'); each is matched word by word against the 27 customers after collapsing official forms ('Управления делами Президента' → 'УДП'), and yields its single best match (the organisation itself rather than its branch). A name that matches nothing is **refused**, never widened to all 27 customers, whose totals would otherwise be credited to it. Without the LLM, customer names are found in the question text by matching runs of words against the 27 names; naming another kind of organisation ('акимат', 'министерство') is refused.
- **Items**: matched to ENSTRU codes in tiers: the name equals the text, contains it as words, every word matches (endings ignored), or trigram similarity ≥ 0.45. Within a tier the most-purchased code wins, so 'бензин' is motor petrol and not 'бензидамин'. Each code is represented by its most common name. Search covers every code sharing the item's name (laptops sit under four codes).

Then exactly one tool runs:

Intent	Tool (agent/tools.py)	Notes
fair_price	fair_price_for_lot(lot_ref)	Needs a lot number; without one the answer asks for it
price_anomaly	price_anomalies(threshold, enstru, text, bins, year, direction, top_k)	Default threshold 30%; direction 'under' for below-market questions
volume_anomaly	volume_anomalies(bins, enstru, text, year, top_k)	Uses analytics/volumes.py

Intent	Tool (agent/tools.py)	Notes
compare	compare_customers(bins, year, enstru, text, unit) / compare_years(years, …)	Per-customer comparison, or per-year when two or more years are named
analytics	price_stats(enstru, unit, year)	Falls back to search when the item is not confidently resolved
search	search_procurements(text, bins, codes, year, top_k, unit)	Default for everything else

Every tool returns a `ToolResult`: tool name, params, method, n_analyzed, summary figures, Top-K rows (each with an ID and portal URL), comparison baselines, DQ figures, limitations, a found flag, and for table-style answers the largest purchases behind the table. SQL is built only from fixed templates with bound parameters.

9.3 Verdict and fact validator (agent/validator.py)

- The LLM receives a system prompt in the answer language, the question, a draft brief and the tool facts as JSON. It writes 1–3 sentences (max 220 tokens, temperature 0, fixed seed, thinking off).
- The validator extracts every number, BIN and date, including Russian and Kazakh formats such as '1 234 567,8', '12,5 млн' and '30%'. Each must match a number in the facts, allowing common derived forms (rounding, ×100 for shares, thousands, millions, billions) within a 0.6% tolerance. Small integers 0–3 are allowed.
- Claim checks then look at how numbers are used: the compared total cannot be presented as the flagged count ('347 series grew' when 347 were compared), a price outside Q1–Q3 cannot be called inside it, and the Q1–Q3 range cannot be called a confidence interval. An organisation's number labelled 'ИНН' is corrected to 'БИН'.
- On failure the LLM gets one corrective retry that names the problem. If that also fails, the template verdict from composer.py is used and verdict_source is set to 'template'.

9.4 Composer (agent/composer.py, i18n.py)

Renders Markdown in Russian or Kazakh with six fixed sections:

#	Section	Source
1	Verdict	LLM (validated) or template
2	Search parameters and data used: period, BINs, filters, item resolution, N	tool result
3	Analytics and comparison: baselines, averages, medians, deviations	tool result
4	Metric: the method in one paragraph	tool result
5	Confidence and limitations: score, DQ rates, applicable limitations	tool result
6	Top-K: object IDs with direct goszakup.gov.kz links	tool result

A refusal keeps the same structure: its parameters show what was asked and that no search was run, and the verdict states the reason.

9.5 LLM client (agent/llm.py)

- Uses Ollama's native `/api/chat`, not the OpenAI-compatible endpoint, because it supports JSON-schema constrained output and switching Qwen3.5's thinking mode off.
- Default model `qwen3.5:9b`; `num_gpu = 99` forces every layer onto the GPU. On an RTX 5070 Laptop (8 GB) this uses about 6.4 GB and is about 25% faster than Ollama's auto-fit.
- `available()` checks /api/tags at start-up; if the model is missing, the agent runs without an LLM.
- Typical latency: about 1.5 s routing, 0.1–2 s tool, 2–4 s verdict. The first request after the model unloads takes about 45 s.

10. API, web chat and worker

Endpoint	Behaviour
GET /	Serves the static web chat (api/static/chat.html)
POST /ask	Body {question: 3–1000 chars}. Returns the full AgentAnswer: markdown, route, tool result, verdict_source, validation, timings. Requests are serialised with an asyncio lock because there is one local GPU. Database errors become HTTP 503.
GET /health	Database status, data_as_of (journal watermark), lag_hours, freshness_sla_ok (lag ≤ 24 h), last ingest run, and LLM model and availability

The **worker** (`tenderai worker`) runs the incremental sync once at start-up, then every 3 hours with APScheduler in the Asia/Almaty timezone. It allows a single instance, coalesces missed runs and logs failures without crashing, so the next tick retries.

11. Deployment

Docker Compose (deploy/docker-compose.yml)

Service	Image	Role
postgres	postgres:16.4-alpine	Database; 1 GB shm, tuned buffers; host port 5434
ollama	ollama/ollama:0.40.3	Local LLM on the NVIDIA GPU; keeps the model loaded 30 min; port 11434
migrate	tenderai:0.1.0	One-off: applies migrations once Postgres is healthy
api	tenderai:0.1.0	FastAPI on port 8010; starts after migrate succeeds
worker	tenderai:0.1.0	Incremental sync every 3 h

All app services share one image built from `deploy/Dockerfile`: Python 3.12 slim, the package installed with pip, running as non-root user 10001, no secrets baked in. Host ports can be changed with PG_HOST_PORT, OLLAMA_HOST_PORT and API_HOST_PORT.

Kubernetes (deploy/k8s/)

Manifest	Contents
00-namespace-secrets.yaml	Namespace, Secret (OWS_TOKEN, DB passwords) and ConfigMap
10-postgres.yaml	Postgres StatefulSet (1 replica) with a PVC, and a Service
20-ollama.yaml	PVC for model weights; Ollama Deployment with nvidia.com/gpu: 1 and 16 Gi memory; Service
30-app.yaml	Migrate Job; API Deployment (1 replica, readiness on /health); Service; incremental CronJob '0 */3 * * *' with concurrencyPolicy Forbid

The API is kept at one replica because there is one local LLM; the LLM must be scaled before the API.

12. Security

- The OWS token exists only in the environment, a Docker or Kubernetes secret, or the gitignored .env. It is a SecretStr in settings and never logged.
- The agent's database role is read-only, limited to core and mart, with a 15 s statement timeout.
- The LLM never writes SQL. Tools use fixed SQL templates with bound parameters.
- Prompt-injection surface is small: user-supplied lot names reach the LLM only inside a JSON facts block, the LLM has no tools, SQL or network access, and its output is validated.
- Containers run as non-root, and .dockerignore excludes .env, data and fixtures from the image.

13. Testing and evaluation

Unit tests (pytest)

Test file	Covers
test_router.py	Intent rules, language detection, hard facts, years, units, direction, off-topic
test_name_matching.py	Splitting customer names, collapsing official forms, word endings
test_validator.py	Number, BIN and date parsing in RU/KZ formats; claim checks; ИНН → БИН
test_composer.py	Six-section rendering, template verdicts, refusals, Top-K examples, quantity check
test_fair_price.py	Fair Price steps with hand-checkable numbers
test_volumes_and_index.py	Volume anomaly rules and the internal price index (mix-effect removal)
test_lot_names.py	Name cleaning

Fixtures in `tests/fixtures/ows` hold sample API responses and a GraphQL schema introspection snapshot, which also supports diffing if OWS changes its schema.

Evaluation (eval/)

Question sets rebuilt from the current data with plain SQL: `build_heldout.py` holds 23 fresh questions written after the agent was finished, on customers, items and wordings used nowhere else; the agent is not changed to pass them. `build_corpus.py` holds 54 hand-picked questions (customers by name, everyday items, lots with a known verdict, refusals and traps) that were also used during development, see eval/corpus.md; `build_golden.py` generates 40 templated questions. `run_eval.py` checks each expected fact; facts the question already contains are not scored, and Fair Price verdicts and refusals are scored as separate checks. `check_leakage.py` verifies that no evaluation question is among the LLM's few-shot examples and no evaluation identifier or figure is in the code. The assignment's target is fact accuracy of at least 85%.

Set and mode	Cases	Fact accuracy	Verdict / refusal checks	Intent accuracy
Held-out, with Qwen3.5 9B (3 runs)	23	86.4% (171 / 198)	100% (21 / 21)	95.7%
Corpus, with Qwen3.5 9B (3 runs)	54	97.3% (438 / 450)	100% (39 / 39)	96.3%
Golden set, with Qwen3.5 9B	40	100% (89 / 89)	—	100%
Held-out, rules only (no LLM)	23	54.5% (36 / 66)	85.7% (6 / 7)	82.6%
Corpus, rules only (no LLM)	54	61.3% (92 / 150)	100% (13 / 13)	94.4%
Golden set, rules only (no LLM)	40	100% (89 / 89)	—	100%

One product is often split into codes that differ only in the plan description (petrol АИ-92 / АИ-95, summer / winter diesel, A4 / A3 paper, battery types). The agent reads these descriptions (mart.enstru_variant): a named variant selects its code, a bare product name selects every variant, and prices are reported per variant. Before this, the held-out set scored 71.0% and the corpus 80.7%. Remaining misses: Kazakh item names transliterated instead of translated ('пияз'), a customer name the LLM also copies into the item field, and

abbreviations ('дизтопливо').

Stability. The LLM is not fully deterministic, so `run_eval.py --repeats 3` asks every question three times, and `build_paraphrases.py` words each mandatory question type (plus one analytics question) 5–7 ways in Russian and Kazakh, all expecting the same facts. Held-out and corpus: identical answers in all three runs. Paraphrases: 24 × 3, 94.8% (219 / 231), identical in all runs; the one miss is the abbreviation 'летнее дизтопливо'.

Without the LLM, free-text item names cannot be turned into product codes, which is why the corpus scores lower in that mode; questions that give a code, a lot number or a BIN are unaffected. The corpus prompted the fixes behind its earlier 100% (first run with the LLM: 66.9%). A review on 14 October found reference answers that treated one variant code as the whole product and few-shot examples that repeated evaluation questions; after correcting both and tightening the scoring the corpus scored 80.7%; telling product variants apart raised it to 97.3%. The held-out set is the honest measure.

14. Risks and limitations

Area	Limitation	Mitigation
Data	Two ENSTRU formats with no mapping table	Exact match, then prefix fallback; comparison level shown
Data	Broad ENSTRU codes: wide price spread within one code	Leave-one-out baseline, robust z confirmation, median and IQR
Data	≈70% of plan lines lack a delivery location	KATO fallback chain with recorded source; stated as a limitation
Data	Services priced per '1 service'	Excluded from unit-price anomalies by default
Data	Volume anomalies use planned, not executed, quantities	Latest plan versions; stated in every volume answer
Data	Short history: 2025 has a one-year baseline	z-test only with ≥ 2 previous years; confirmed tier
Data	Only 27 customers, so peers are the other 26	N and level shown; more BINs is a config change
Data	CPI bulletins cover 01.2024–09.2026 only	Nearest covered month used; add new bulletins and re-run load-cpi
API	Token expiry or revocation (403)	Clear error; /health shows last ingest status
API	Transient timeouts and 5xx	Backoff, idempotent upserts, per-BIN reconciliation
API	Journal delays	2 h overlap; a nightly backfill re-run reconciles
API	Journal volume spikes (4.6 M changes on one day)	Deletions only (action=D); progress logged; a sync still fits the 3 h cycle
AI	Hallucinated figures	LLM never computes; validator with one retry, then template
AI	Misclassified intent or wrong item match	Hard-fact overrides; tiered item matching; resolved code shown to the user
AI	Wrong customer (unknown or similar names)	One best match per name; an unknown name is refused, never widened to all 27
AI	Right number, wrong meaning in the verdict	Claim checks with a corrective retry, then the template verdict
AI	Weaker Kazakh in a 9B model	Fixed Kazakh templates; only the verdict is generated
AI	One 8 GB GPU	Requests serialised; template answers if the LLM is down

> A price 'anomaly' is a signal for review, not proof of a violation. The Fair Price band reflects what the other 26 customers actually paid; it is not an external market benchmark.

15. Operating the system

First run (local)

```
cp .env.example .env                       # set OWS_TOKEN
docker compose -f deploy/docker-compose.yml up -d postgres ollama
docker compose -f deploy/docker-compose.yml exec ollama ollama pull qwen3.5:9b
python -m venv .venv && .venv/Scripts/pip install -e ".[dev]"
python -m tenderai.cli migrate
python -m tenderai.cli backfill            # ~25 min
python -m tenderai.cli load-cpi cpi_dataset
python -m tenderai.cli etl                 # ~1 min
python -m tenderai.cli serve               # http://127.0.0.1:8010
python -m tenderai.cli worker              # incremental sync every 3 h
python eval/build_corpus.py && python eval/run_eval.py eval/corpus.jsonl
```

CLI reference

Command	Purpose
migrate	Apply SQL migrations from tenderai/db/sql
backfill [--bins] [--entities] [--no-refs] [--no-subjects]	Full load of the target BINs for 2024–2026
incremental [--no-etl]	Changes since the last run plus journal deletions, then rebuild core and marts
etl [--no-marts]	Rebuild core and marts from raw
load-cpi <path>	Load official CPI from a folder of bulletins, or a CSV
worker	Incremental sync every 3 hours
ask "..." [--no-llm] [--json]	Ask from the terminal; --json prints the full structured answer
serve [--host] [--port]	Run the HTTP API and web chat (default 127.0.0.1:8010)

Configuration (environment variables)

Variable	Default	Meaning
OWS_TOKEN	(required)	Bearer token for the OWS API
OWS_BASE_URL	https://ows.goszakup.gov.kz	API base URL
OWS_CONCURRENCY / OWS_PAGE_SIZE	4 / 200	Parallel requests and GraphQL page size
DATABASE_URL	tenderai @ localhost:5434/tenderai	Owner role for ingest and ETL
AGENT_DATABASE_URL	tenderai_agent @ localhost:5434/tenderai	Read-only role for agent tools
LLM_BASE_URL / LLM_MODEL	http://localhost:11434 / qwen3.5:9b	Ollama server and model
LLM_NUM_GPU	99	Layers to place on the GPU

Monitoring

- `GET /health`: database, LLM availability, lag_hours and the 24-hour freshness flag.
- `ops.ingest_run` and `ops.ingest_count`: run history and API total versus rows fetched per BIN.
- `mart.dq_summary`: data-quality issue counts by entity and rule.
- `ollama ps`: confirms the model is fully on the GPU (PROCESSOR column).